

# ***Prime Throughput Performance Evaluation***

## **Source Code and GitHub Repositories**

The files used for this evaluation can be found at the following repository:

<https://github.com/sam444555/cs2910>

The original Prime code used is available at the following Johns Hopkins DSN repository:

<https://github.com/jhu-dsn/prime>

## **Host System Specifications**

The throughput evaluation was performed locally using Docker on a system with the following specifications:

- OS: Windows 11
- Intel Core i7-14700KF
  - 20 cores: 8 P-cores, 12 E-cores
  - 3.4 GHz base clock speed, up to ~5.6 GHz boost
- Memory: 32 GB DDR5 @ 4800 MT/s

## **Docker Configuration**

The Docker files & setup guide can be found at:

<https://github.com/sam444555/cs2910/tree/main/prime/docker>

6 Prime replicas were deployed across six Docker containers, satisfying Prime's minimum replica requirement of  $3f + 2k + 1$  ( $f=1, k=1$ ) as defined in

<https://github.com/sam444555/cs2910/blob/main/prime/src/def.h> by the **NUM\_SERVERS** macro.

Each container ran a Prime replica and a Spines process. One container additionally ran the driver program used to emulate additional clients. Each replica was assigned a unique positive integer identifier starting at 1.

For the throughput testing, the containers were named **prime1, prime2, prime3, prime4, prime5, prime6**, with the driver program running on **prime1**. The six containers

communicated over a custom Docker bridge network. The complete Docker topology can be found in Figure 1. The containers use **Ubuntu 22.04** as their operating system.

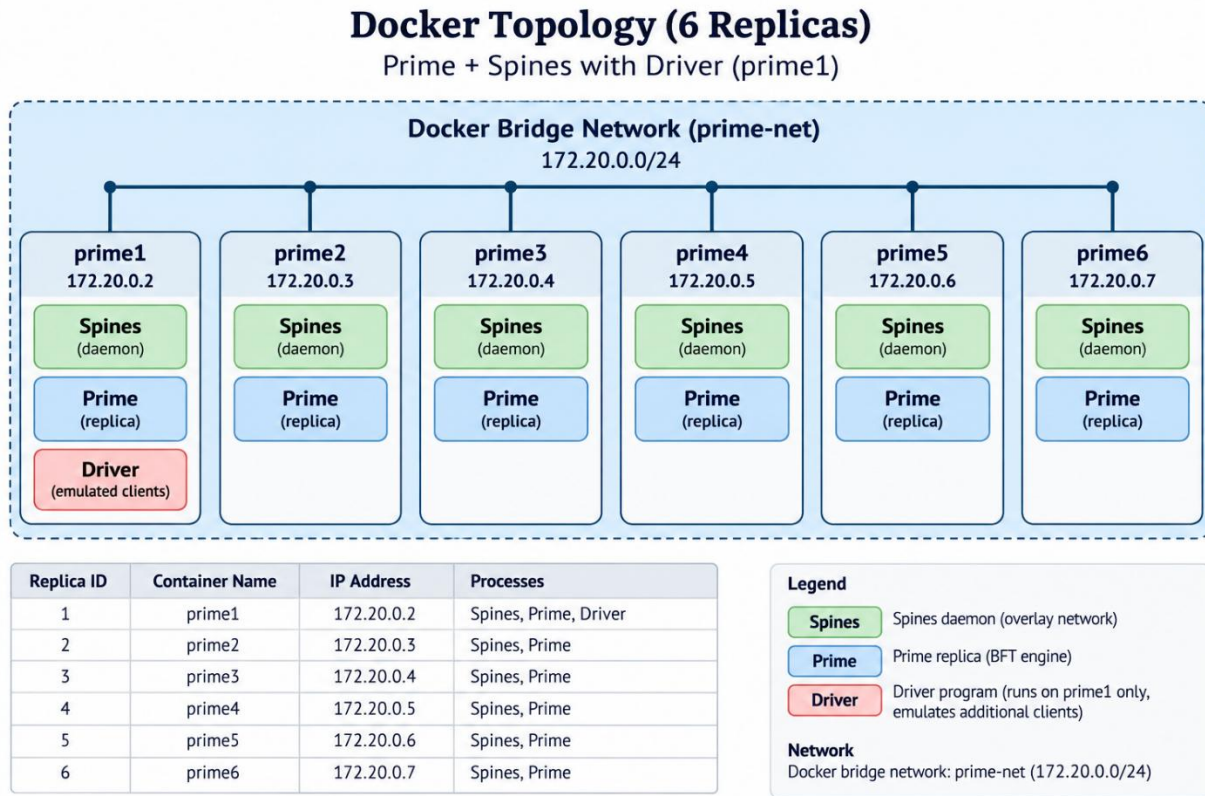


Figure 1. Docker network topology

## Driver Setup

The driver program used for the testing can be located at:

<https://github.com/sam444555/cs2910/blob/main/prime/src/driver.c>

The original **driver.c** program did not include throughput measurement, so it had to be modified to include this functionality. Throughput is calculated using the following

$$\text{equation: } \textit{Throughput} = \frac{\textit{Total Updates Completed}}{\textit{Elapsed Time (seconds)}}$$

**Elapsed time** is calculated using the throughput stopwatch (throughput\_sw in driver.c) which starts in Run\_Client() and stops in Process\_Message() once all updates have been successfully processed.

**Total Updates Completed** is represented by the needed\_count variable and is input into the program using the -c (benchmark count) flag.

Another modification made to driver.c was to pass the number of emulated clients using the -n flag, replacing the macro definition in order to eliminate the need to recompile the program every time this value changes. It is now stored in the num\_clients\_to\_emulate variable.

Additional changes were made; however, these are discussed later in the document to provide the appropriate context.

### To run the driver:

```
./driver -l <LOCAL_IP> -i <CLIENT_ID> -s <SERVER_ID> -c <UPDATE_COUNT> -n  
<EMULATED_CLIENTS>
```

- LOCAL\_IP: IP address of the machine the driver program is running on
- CLIENT\_ID: A positive integer starting from 1 which uniquely identifies the client process
- SERVER\_ID: The ID of the prime replica the client process is communicating with.
- UPDATE\_COUNT: The total number of updates the driver program will send
- EMULATED\_CLIENTS: Number of clients being emulated by the driver program, which also determines the maximum number of outstanding updates

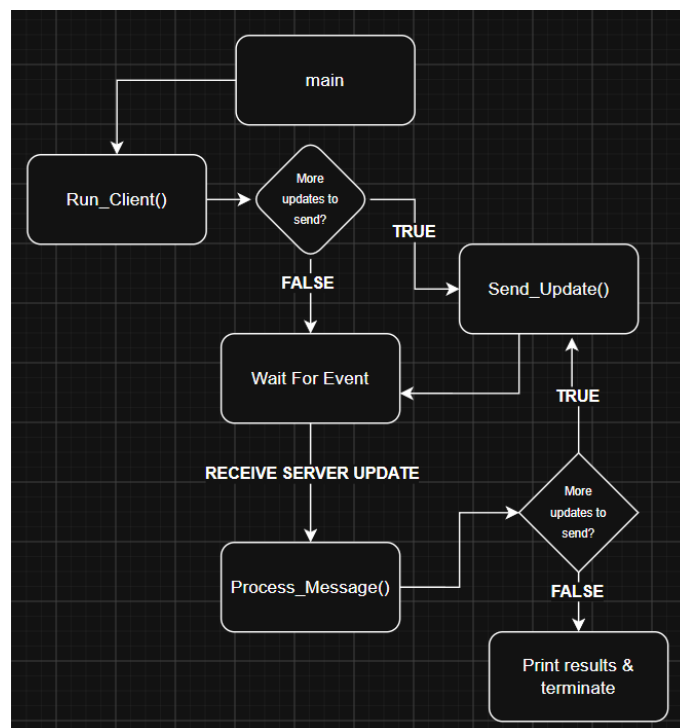


Figure 2. Driver.c high level control flow

## **Testing: No Throughput Constraint**

In the original driver's code, once the number of emulated clients exceeded ~20, the driver would unpredictably stall indefinitely across runs. In the driver's `Send_Update()` function, its `sendto()` call was blocking. At approximately the ~20-client threshold, the `sendto()` call would sporadically block and fail to return control to the driver, causing the process to stall permanently.

I changed the `sendto()` call to nonblocking and created an error handler for failed sends. Upon detecting a failure to send, the handler would roll back changes made by the `Send_Update()` function (specifically the `time_stamp` increment which is used for driver update sequence numbers), record the failure to send (tracked by the global variable `failed_sends`), and return control to the event handler (`E_handle_events()`).

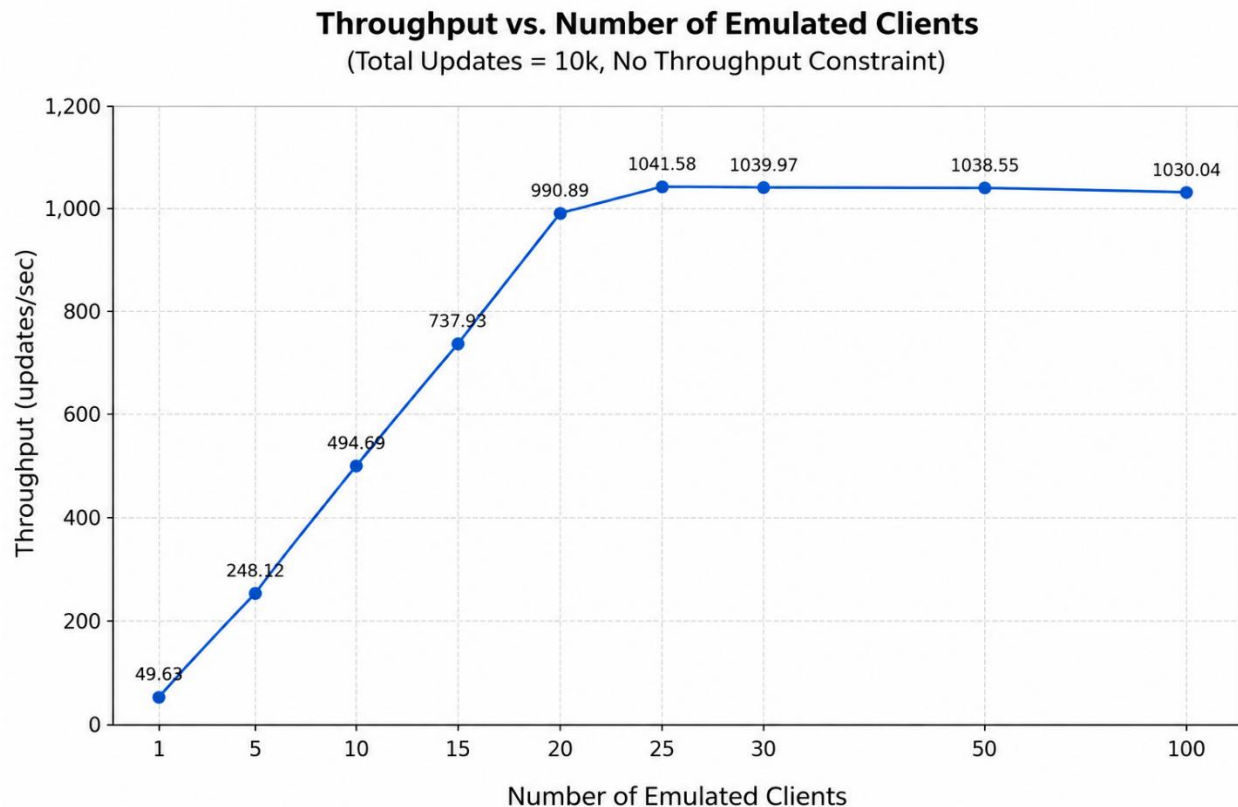
This modification reduced the number of stalls, but did not eliminate them entirely. Without a deeper understanding of Prime, I cannot say with 100% certainty what the cause of this behavior is. However, based on debugging and testing, my initial leading theory was that the stalls were due to the driver's socket's outgoing queue becoming full due to higher throughput and Prime not draining it fast enough.

When `Send_Update()` performs a successful send (i.e. `sendto()` does not return an error), the function increments the number of outstanding updates (the number of updates sent to Prime for which a response has not yet been received). During stalls, however, the value of outstanding updates was nonzero, meaning the driver "thought" it had successfully sent >0 updates. Under normal operation, the event handler would eventually call `Process_Message()` when Prime responds to these outstanding messages. But this behavior never occurred, resulting in the freeze.

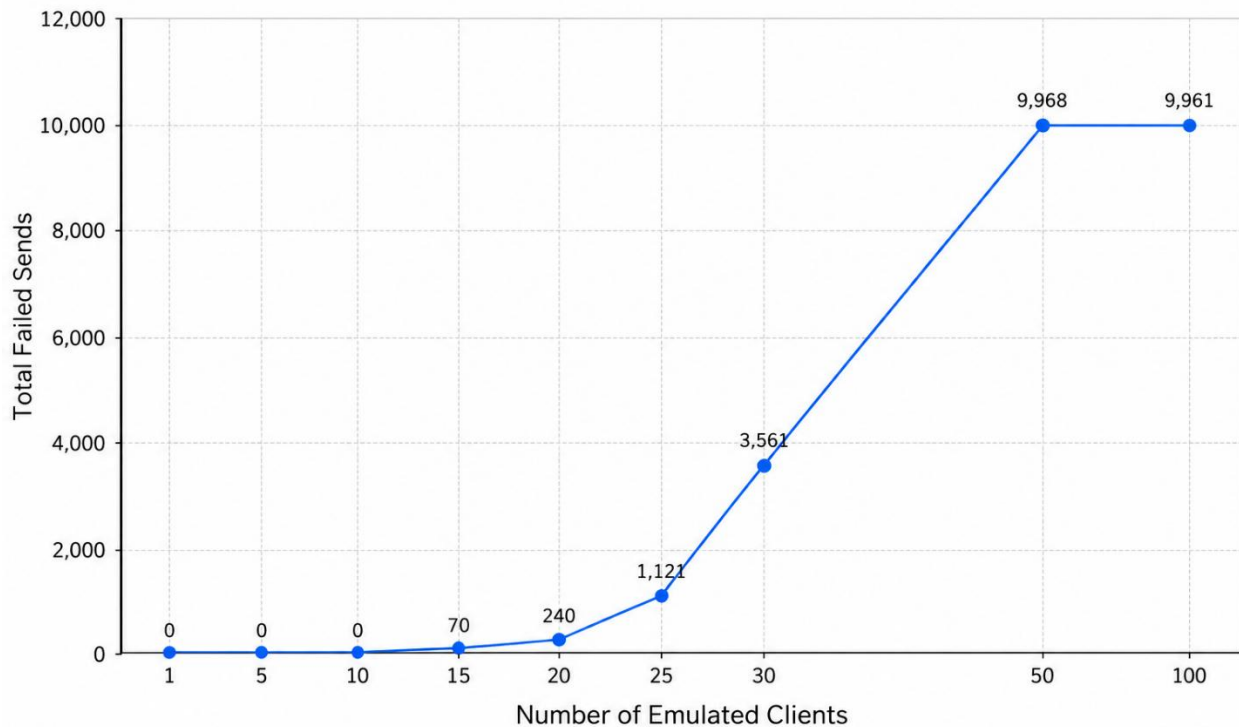
Therefore, my initial theory is likely incorrect, and the most likely explanation for the stalling is Prime's queuing behavior and its decision to reject packets when it becomes saturated. The driver's current socket configuration for IPC uses a Unix-domain datagram socket, which does not guarantee data delivery. There is an option to disable IPC communication in `prime/src/def.h` by setting the `USE_IPC_CLIENT` macro to 0. However, I encountered difficulties establishing the TCP connection between the driver and Prime, so this configuration was not evaluated further.

The following tests were conducted with no throughput constraint applied (e.g. no latency or bandwidth limitations). These results serve as the best-case baseline for Prime's throughput performance.

| No Throughput Constraint Tests (total updates = 10k) |                     |                    |                          |
|------------------------------------------------------|---------------------|--------------------|--------------------------|
| Number of Emulated Clients                           | Time Elapsed (secs) | Total Failed Sends | Throughput (updates/sec) |
| 1                                                    | 201.48              | 0                  | 49.63                    |
| 5                                                    | 40.3                | 0                  | 248.12                   |
| 10                                                   | 20.21               | 0                  | 494.69                   |
| 15                                                   | 13.55               | 70                 | 737.93                   |
| 20                                                   | 10.09               | 240                | 990.89                   |
| 25                                                   | 9.6                 | 1121               | 1041.58                  |
| 30                                                   | 9.62                | 3561               | 1039.97                  |
| 50                                                   | 9.63                | 9968               | 1038.55                  |
| 100                                                  | 9.71                | 9961               | 1030.04                  |



**Total Failed Sends vs. Number of Emulated Clients**  
(Total Updates = 10k, No Throughput Constraint)



The throughput peaked in the range of 1,000-1,040 updates/sec. It began to stagnate once the number of emulated clients exceeded ~25. Anything beyond this value resulted in no additional benefit and led to an explosion in cumulative sending failures.

Therefore, I cannot conclusively say that Prime's "best-case" baseline is ~1,040 updates/sec due to the instability observed as the system approaches this level of throughput. If my theory that the instability stems from Prime itself discarding updates due to overflow is correct, employing a transport protocol with reliable delivery guarantees (such as TCP) could potentially resolve this issue. However, this additional overhead would likely result in a lower throughput. The significance of the reduction is difficult to determine given I was unable to get TCP to function.

**In practice, the highest consistently stable throughput observed during testing occurred with 15-16 emulated clients, corresponding to roughly 740-800 updates/second.**

## **Testing: Bandwidth & Latency Constraints**

| <b>0ms (~.067 ms avg) RTT, 100 Mbps Link Speed</b> (total updates = 10k) |                            |                           |                                 |
|--------------------------------------------------------------------------|----------------------------|---------------------------|---------------------------------|
| <b>Number of Emulated Clients</b>                                        | <b>Time Elapsed (secs)</b> | <b>Total Failed Sends</b> | <b>Throughput (updates/sec)</b> |
| 1                                                                        | 207.32                     | 0                         | 48.24                           |
| 5                                                                        | 41.38                      | 0                         | 241.64                          |
| 10                                                                       | 21.93                      | 0                         | 456                             |
| 15                                                                       | 17.03                      | 81                        | 587.1                           |
| 20                                                                       | 13.86                      | 173                       | 721.6                           |
| 25                                                                       | 14.03                      | 684                       | 712.57                          |
| 30                                                                       | 14.04                      | 2684                      | 712.32                          |
| 50                                                                       | 13.26                      | 9968                      | 753.96                          |
| 100                                                                      | 13.42                      | 9968                      | 744.93                          |

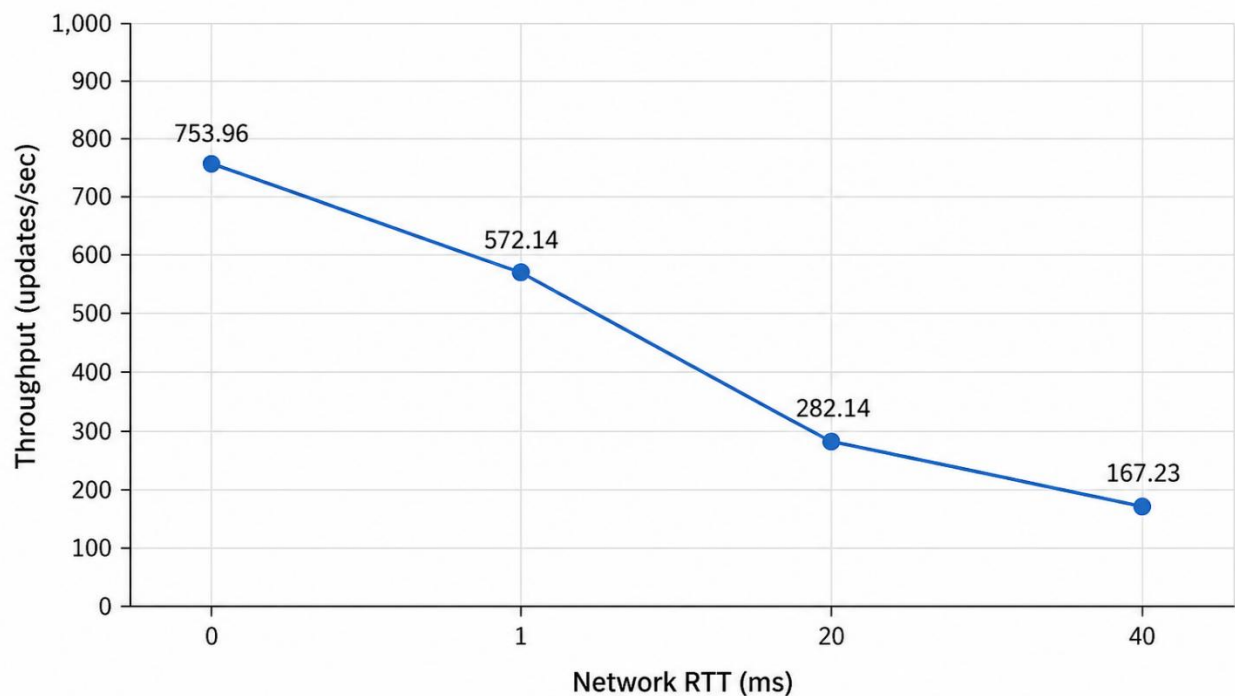
| <b>1 ms RTT, 100 Mbps Link Speed</b> (total updates = 10k) |                            |                           |                                 |
|------------------------------------------------------------|----------------------------|---------------------------|---------------------------------|
| <b>Number of Emulated Clients</b>                          | <b>Time Elapsed (secs)</b> | <b>Total Failed Sends</b> | <b>Throughput (updates/sec)</b> |
| 1                                                          | 215.32                     | 0                         | 46.44                           |
| 5                                                          | 47.22                      | 0                         | 211.76                          |
| 10                                                         | 29.3                       | 0                         | 341.28                          |
| 15                                                         | 23.07                      | 61                        | 433.44                          |
| 20                                                         | 19.94                      | 167                       | 501.5                           |
| 25                                                         | 17.48                      | 932                       | 572.14                          |
| 30                                                         | 18.8                       | 3113                      | 531.86                          |
| 50                                                         | 19.05                      | 9968                      | 524.89                          |
| 100                                                        | 19.2                       | 9973                      | 520.97                          |

| <b>20 ms RTT, 100 Mbps Link Speed</b> (total updates = 10k) |                            |                           |                                 |
|-------------------------------------------------------------|----------------------------|---------------------------|---------------------------------|
| <b>Number of Emulated Clients</b>                           | <b>Time Elapsed (secs)</b> | <b>Total Failed Sends</b> | <b>Throughput (updates/sec)</b> |
| 1                                                           | 840.5                      | 0                         | 11.9                            |
| 5                                                           | 170.27                     | 0                         | 58.73                           |
| 10                                                          | 83.81                      | 0                         | 119.31                          |
| 15                                                          | 54.6                       | 0                         | 183.13                          |
| 20                                                          | 42.52                      | 0                         | 235.18                          |
| 25                                                          | 35.44                      | 3                         | 282.14                          |
| 30                                                          | 35.68                      | 48                        | 280.29                          |
| 50                                                          | 35.4                       | 9959                      | 281.98                          |
| 100                                                         | 35.69                      | 9963                      | 280.2                           |

| <b>40 ms RTT, 100 Mbps Link Speed</b> (total updates = 10k) |                            |                           |                                 |
|-------------------------------------------------------------|----------------------------|---------------------------|---------------------------------|
| <b>Number of Emulated Clients</b>                           | <b>Time Elapsed (secs)</b> | <b>Total Failed Sends</b> | <b>Throughput (updates/sec)</b> |
| 1                                                           | Not Measured               | Not Measured              | Not Measured                    |
| 5                                                           | 293.41                     | 0                         | 34.08                           |
| 10                                                          | 147.07                     | 0                         | 67.99                           |
| 15                                                          | 97.5                       | 1                         | 102.56                          |
| 20                                                          | 74.21                      | 0                         | 134.75                          |
| 25                                                          | 61.93                      | 6                         | 161.48                          |
| 30                                                          | 61.7                       | 98                        | 162.08                          |
| 50                                                          | 59.8                       | 9961                      | 167.23                          |
| 100                                                         | 60.83                      | 9965                      | 164.38                          |

### Peak Throughput vs. Network RTT

(Total Updates = 10k, 100 Mbps Link Speed)



Note: The stalling issues occurred, albeit far less frequently, in these tests as well. A reliable delivery guarantee mechanism should be incorporated to ensure consistent system stability.